SE1020 · OBJECT ORIENTED PROGRAMMING

Book & Bloom

Online Bookstore Management System

Java · Spring Boot · JSP · Jakarta Servlets · Text-file persistence

IT25102131

IT25102114

IT25102130

IT25101966

IT25102132

IT25102519

Project report · Version 1.0.0 · 28 September 2026

Student names and genuine GitHub contribution evidence must be completed by the team before submission.

1. Abstract

Book & Bloom is a Java web application for browsing and managing an online bookstore. Customers can discover books, maintain accounts, place demonstration orders and write reviews. Administrators maintain books, authors, categories, customer details, order status and review moderation. The implementation uses Spring Boot for application startup and dependency injection, JSP for page delivery, an explicit Jakarta Servlet for API requests, and Java file operations for persistent storage. Six modules each implement all four CRUD operations. The project emphasizes explainable OOP, reliable data changes and a responsive interface.

2. Objectives and scope

The application applies encapsulation, inheritance, polymorphism, abstraction and information hiding to a practical business domain. It provides more than the minimum three CRUD operations and at least three user interfaces. It demonstrates file read/write instead of database storage, and separates presentation, HTTP routing, business rules and persistence so each student can explain an assigned flow.

Scope includes catalog discovery, two book formats, customer accounts, a browser bag, order creation and state transitions, authors, categories and reader reviews. The store uses LKR. Printed orders have a LKR 350 delivery fee below LKR 5,000; digital-only orders and qualifying orders have no delivery fee. This is an academic demonstration: payments, shipping and digital-content fulfillment are not external integrations. Original fictional books avoid dependence on licensed e-book content.

3. Interpretation of the supplied brief

The project description mentions Spring Boot in its technology list and JSP/Servlets in its sample contribution. The user selected **Spring Boot + JSP/Servlets**. Both are present: Spring Boot runs embedded Tomcat, `PageController` resolves JSP views, and `ApiServlet extends HttpServlet` handles API requests.

The overview requires file storage, while another line allows MySQL/SQL. This implementation selects **file read/write**, matching the file-handling marking criterion and viva emphasis. No database dependency is present. Executable WAR packaging supports JSP compilation.

The sample workload is an Online Library Management System example, not a requirement to build borrowing/returns. The selected project is a bookstore; orders replace borrowing. Category management provides a coherent sixth module alongside users, books, orders, authors and reviews.

4. Requirements and marking traceability

Criterion	Marks	Implementation evidence
Assigned CRUD functionality	30	Six dedicated services and matching customer/admin interfaces; constrained deletion preserves relationships
OOP concepts	20	Abstract `Book` and `User`; subclass overrides; repository interface; private fields; DTO-like safe profiles
File handling	10	`FileStoreRepository`: read UTF-8 JSON text, detached snapshots, temporary write, force, atomic replace, lock
UI design	10	Nine JSP views, responsive layout, search/filter/sort, dialogs, empty/error states, keyboard focus, local artwork
Individual contribution / GitHub	10	Allocation and evidence template provided; genuine student commits and repository history must be added
Presentation / viva	10	Demo script and backend question guide supplied; actual student explanation must be performed
Documentation	10	This report, class diagram, workload map, API guide, tests and setup instructions
Total	**100**	Marks depend on assessment of actual work and understanding

The submission package cannot truthfully contain names or commit histories that have not been supplied or created by the students. Those are explicitly pending instead of fabricated.

5. Architecture

```
Browser: HTML / CSS / JavaScript
    | page request                  | JSON API + session + CSRF
PageController -> JSP views      ApiServlet
                                    |
             User / Book / Author / Category / Order / Review services
                                    |
                          StoreRepository interface
                                    |
                               FileStoreRepository
                                    |
                        data/bookstore.txt (UTF-8)
```

`PageController` selects page templates. The templates share a header, footer and accessible dialog structure. JavaScript retrieves data and renders each page; input values are escaped before being inserted into markup. `ApiServlet` reads bounded JSON input, checks CSRF, extracts the server-side session identity, invokes a service and writes a JSON response. Services enforce business rules on every call, regardless of UI controls. The repository alone manages files.

6. OOP implementation

Encapsulation

Domain attributes are private and accessed through getters/setters. Services centralize validation instead of trusting HTML constraints. For example, `BookService.save` validates price, stock, referenced author/category, format and unique ISBN before committing.

Inheritance

`PrintedBook` and `EBook` extend `Book`, sharing catalog fields. `AdminUser` and `Customer` extend `User`, sharing identity, profile and credential fields. This avoids duplicated definitions while allowing subtype-specific behavior.

Polymorphism

`OrderService` holds a `Book` reference and calls `requiresShipping()`. `PrintedBook` returns true and `EBook` returns false. The same checkout flow therefore calculates the correct delivery rule without hard-coding subtype checks in that calculation. `Rules.admin` calls `canManageCatalog()` on a `User`: the runtime type determines permission. Jackson type tags restore the appropriate concrete class from text storage.

Abstraction

`StoreRepository` exposes `snapshot()` and `transaction()` without exposing paths, file channels or serialization. Services depend on that interface. `Book` and `User` specify abstract behavior that each concrete subclass must implement.

Information hiding

`FileStoreRepository` keeps its state, mapper, file and lock private. It returns detached data rather than the live stored object graph. Password derivation is hidden in `Passwords`; public API profiles deliberately omit `passwordHash`. Passwords are salted PBKDF2-HMAC-SHA256 hashes with 210,000 iterations, not plaintext or reversible encryption.

7. Domain and persistence design

The editable diagram is in `CLASS_DIAGRAM.md`; the standalone visual is `class-diagram.svg`.

Collection	Main properties	Integrity rules
Users	ID, kind, name, unique email, hash, address	Public registration creates customers; protected demo administrator; accounts with order history cannot be deleted
Books	ID, type, title, author/category IDs, ISBN, price, stock	Positive price, nonnegative stock, valid references and unique ISBN
Authors	ID, name, biography	Unique name; cannot delete while referenced by books
Categories	ID, name, description	Unique name; cannot delete while referenced by books
Orders	ID, user ID, line snapshots, totals, address, status	Ownership and state transitions; server-computed amounts
Reviews	ID, user/book IDs, rating, comment	One per user/book; owner edits, owner/admin deletes

`bookstore.txt` contains a schema version and six arrays. JSON is a text format, so Java file operations satisfy the file-storage requirement while supporting spaces, quotes, Unicode and newlines safely. A single snapshot is used intentionally: separate `books.txt` and `orders.txt` writes could disagree if the application stopped between writes.

Transaction sequence:

1. Obtain the repository monitor (`synchronized`).
2. Deep-copy the last committed state.
3. Apply validation and mutation to the candidate.
4. Serialize to a temporary file in the same directory.

5. Flush bytes with `FileChannel.force(true)`.

6. Replace the target with `ATOMIC_MOVE`.

7. Publish a detached copy as the current state.

Validation failures and failed atomic replacement do not publish the candidate. A process-level `FileLock` prevents two running servers from sharing the directory. The implementation fails instead of silently weakening atomic replacement on unsupported filesystems. Corrupt existing files fail startup and are preserved. This is a small single-instance store; full-file copies and scans are not designed for production-scale datasets. Atomic rename and flush improve reliability but do not constitute a multi-node database or guarantee against every hardware failure.

8. Functional flows

Discovery: Home → catalog → filter/search/sort → book details → add to bag. The bag contains book IDs and quantities in local storage; it holds no credentials. Price and stock are refreshed from the server, and checkout recalculates them again.

Accounts: Register → sign in → profile read/update → optional password change using the current password → sign out. Administrators can edit customer profiles but cannot use the profile endpoint to change another person's password. Deletion cascades reviews but is blocked if order history exists.

Checkout: Authenticate → validate 1–50 unique lines, each quantity 1–20 → check stock → calculate totals with `BigDecimal` → validate shipping address when required → decrement stock and create order in one transaction.

Order lifecycle: `PLACED` → `PROCESSING` → `COMPLETED`. `PLACED` and `PROCESSING` can move to `CANCELLED`; customers can only cancel their own placed orders. Cancellation restores stock once. Terminal states cannot transition again. Only cancelled orders can be deleted.

Administration: View live counts → search records → create/edit/delete books, authors, categories and customers → process orders → moderate reviews. A book in an active order cannot be removed. Historical order line snapshots remain readable after eligible catalog deletions.

9. UI design

The visual identity uses forest green, warm cream, restrained botanical illustration and editorial serif headings. Original CSS-generated book covers avoid external image dependencies. The interface includes responsive grids, consistent spacing, search and filter controls, cart badges, progress/status badges, clear validation messages and confirmation dialogs. Empty states guide readers toward useful actions. Dialogs are native HTML dialogs with a close button and Escape support. Inputs have labels, focus styles are visible, and a skip link leads to main content. Reduced-motion preferences are respected.

Nine JSP pages: home, catalog, book detail, cart/checkout, login, registration, account, orders and administration. Admin modules have distinct listing, create and edit form states. The JavaScript requirement is explicit through a noscript notice.

10. Validation, safety and testing

Session identities are held on the server. Login changes the session ID and rotates the CSRF token. Mutations require the token. HttpOnly/SameSite cookies, same-origin content rules, anti-framing headers and escaped user-controlled text reduce common browser risks. Business permissions and ownership are checked inside services. Requests are bounded to 64 KiB, with structured errors. The default server is local-only.

The automated suite and HTTP verification are described in `TEST_REPORT.md`. Tests cover rollback, file reload, concurrent stock requests, authorization, account hashing, CRUD, relationships, state transitions and ownership. A

separate HTTP smoke script starts a temporary store, compiles all JSP views and verifies restart persistence. Tests do not use the normal demo data directory.

11. Installation and demonstration

Use JDK 17/21 and the provided Maven wrapper. Import `pom.xml` into IntelliJ, run `mvnw.cmd package`, and launch `java -jar target/book-and-bloom.war`. Visit `http://127.0.0.1:8080`. The Windows helper `start-bookstore.cmd` locates an installed JDK. First-time Maven import requires Internet access; a packaged app uses local assets and local text data.

Use the two public demo accounts listed in README. Show one module's complete CRUD flow, then trace the backend code, inspect `data/bookstore.txt`, and restart to prove persistence. Follow the detailed sequence in `VIVA_GUIDE.md`.

12. Limitations and future work

The demonstration does not implement real payment processing, shipping integration, email verification, password recovery, production-grade abuse controls or licensed digital downloads. The dataset is small, the repository is single-writer and full-file based, and the JavaScript UI is required. A real deployment would need updated dependency/security review, HTTPS, private credentials, operational backups and a different scalability design. None of these external integrations is claimed as implemented.

New-arrival ordering follows insertion order in the file. Updates should preserve that position; it is not a chronological sales analytics system. Dashboard order value is explicitly labelled as order value, not recognized revenue.

13. Contribution evidence and submission completion

See `WORKLOAD_AND_GITHUB.md` for the six IDs, service ownership, CRUD demonstrations and UI states. Add real names, GitHub URL, actual commit history and evidence of each member's changes before submitting. The viva is individual: members should be able to explain their own code and shared infrastructure. Report any assistance according to institutional rules.

14. Source material and references

• Supplied `SE1020 - Prorata Project Topic List.pdf`: topic 6, Online Bookstore Management System.

• Supplied `SE1020 - Project Description.pdf`: Java web application, OOP, minimum CRUD/UI, file storage, deliverables and backend-focused viva.

• Supplied `SE1020 - Sample Project Workload Distribution.pdf`: example module ownership, CRUD breakdown and OOP mapping.

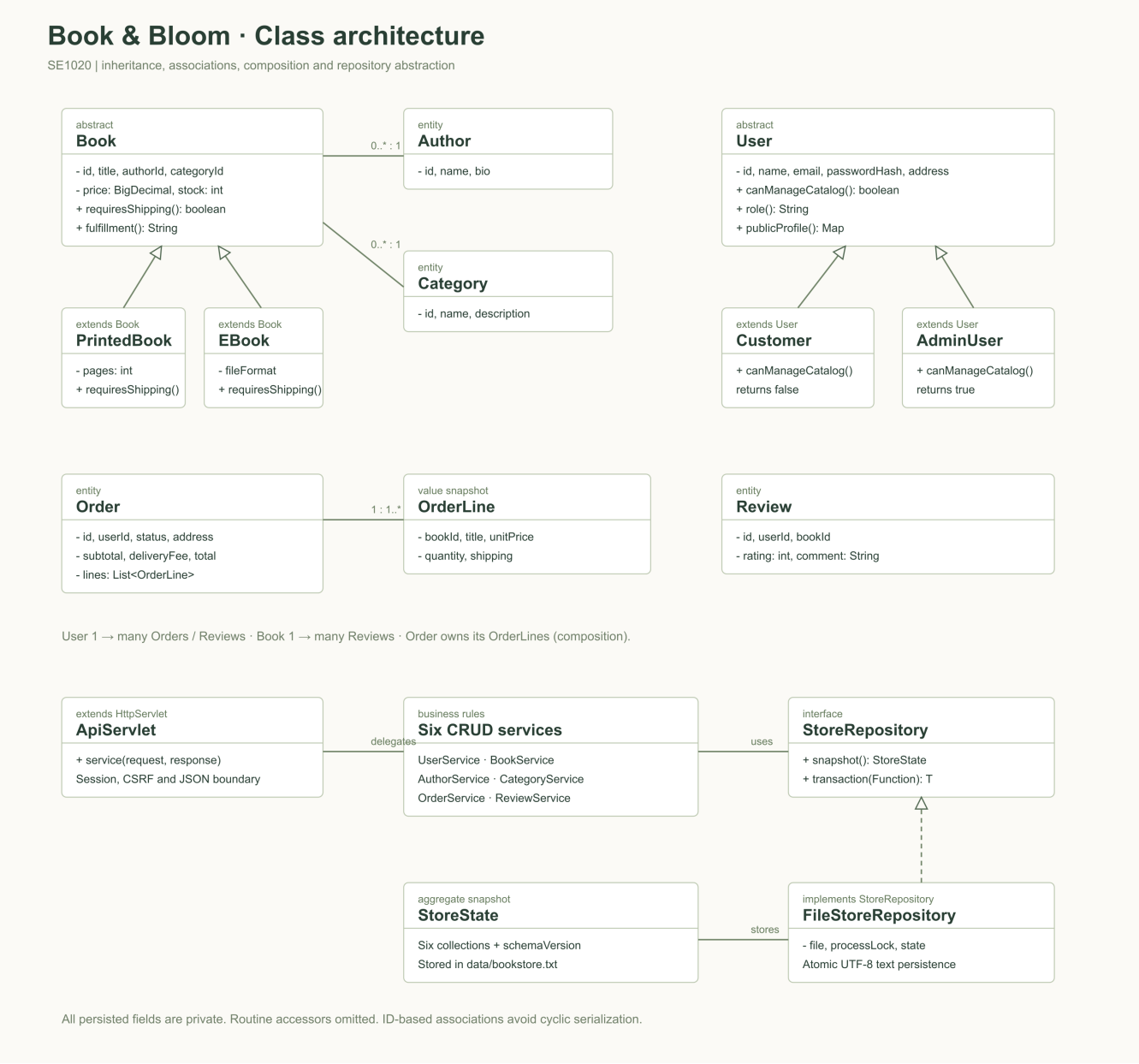
• Supplied `Marking Guide.pdf`: 30/20/10/10/10/10/10 criterion allocation.

• Spring Boot documentation: Servlet web applications and JSP support.

• Spring Boot documentation: Traditional WAR deployment.

Appendix A · Class architecture

Editable sources: docs/CLASS_DIAGRAM.md and docs/class-diagram.svg.



Appendix B · Verification

• **18 JUnit tests passed:** zero failures, errors or skips. Tests use isolated temporary file repositories.

• **32 packaged-application HTTP checks passed:** real server startup, requests, JSP compilation and process restart with an isolated temporary store.

• **Executable WAR packaging succeeded** using the supplied Maven wrapper after stopping the earlier process that held a Windows file lock.

• **JavaScript syntax check passed** using Node `--check`.

• **Browser checks passed:** storefront artwork/layout, catalog search, add-to-bag, reader/admin sign-in, admin dashboard, inventory table and book-creation form were exercised against the running application.

• **Mobile checks passed:** home and catalog at a 390px viewport override (375px content area); document scroll width equalled client width, with no horizontal page overflow. E-book filtering correctly returned the two digital editions.

• **Encoding regression fixed:** JSP fragment now declares UTF-8; all nine page responses are checked for correct text encoding.

• **Format regression fixed:** the book-list endpoint uses a typed Jackson writer so PRINTED/EBOOK tags survive root-list serialization. Both API and checkout classification have regression checks.

Team module allocation

IT25102131 - Users / customer accounts

IT25102114 - Books / inventory

IT25102130 - Orders / checkout

IT25101966 - Authors

IT25102132 - Categories

IT25102519 - Reviews / moderation

Submission completion

Add actual student names, the GitHub repository link and genuine per-member commit evidence. Rehearse each assigned CRUD flow and the backend explanation using docs/VIVA_GUIDE.md. This allocation is a plan, not a claim of past individual contribution.